El mutex de los bloques de Simulink

Por qué no causaba el bloqueo del enlace USB–SPI
y qué se cambió para eliminarlo

Haniel Ulises
Servicio social

11 de octubre de 2026

Resumen

Se responde la pregunta de si el mutex de los bloques de Simulink `escribirPic` y `leerPic` es compatible con Windows y si es la causa del bloqueo de la comunicación con el dsPIC33CK64MC105. El mutex es un objeto del núcleo de Windows (`CreateMutex`, `WaitForSingleObject`, `ReleaseMutex`) y es compatible con ese sistema, pero no puede causar el bloqueo: Simulink ejecuta los bloques uno después del otro en un solo hilo, por lo que el mutex siempre está libre cuando un bloque lo pide. El bloqueo y las tramas corruptas tenían su origen en el firmware del dsPIC, que podía quedarse en una espera activa y dejaba de recibir al desbordarse la FIFO del SPI, y en el uso que la PC hacía del bus, con una escritura y una lectura separadas en cada paso. Se describen los cambios realizados para eliminar esas causas: un protocolo de una sola transferencia full-duplex de ocho bytes por paso, delimitada por `CS` y protegida con número de secuencia y CRC-8; un firmware nuevo que atiende el SPI con DMA; bloques de Simulink en los que un solo bloque usa el FT2232H, sin mutex; y herramientas para probar el enlace sin MATLAB. En Windows, el modelo `MergedPic.slx` del asesor, sin modificaciones y con los bloques nuevos, ejecutó 10 001 pasos a 1 kHz sin errores de comunicación.

Índice

1. La pregunta y la respuesta corta **3**

2. Cómo funcionaban los bloques anteriores **3**

3. El mutex **4**
3.1. Es compatible con Windows 4
3.2. No puede causar el bloqueo 4
3.3. Qué hacía realmente 5

4. Las causas reales del bloqueo **5**

5. El protocolo nuevo **6**

6. El firmware del dsPIC: `dsPicDaq` **7**

7. Los bloques nuevos de Simulink **8**

8. Herramientas de prueba sin Simulink **10**

9. Resultados **10**

10.Pendientes **11**

1. La pregunta y la respuesta corta

La pregunta fue si el mutex que usan los bloques de Simulink es compatible con Windows y si el bloqueo de la comunicación con el dsPIC se debía a él.

- **¿Es compatible con Windows?** Sí. No es un mecanismo de Simulink ni de C++, sino un objeto del núcleo de Windows creado y usado con las funciones de `kernel32`. Compila con MSVC y funciona en cualquier versión de Windows; de hecho, sólo existe en Windows.
- **¿Es la causa del bloqueo?** No. En una simulación de Simulink los bloques se ejecutan en un solo hilo, de modo que nunca hay dos bloques esperando el mutex al mismo tiempo. Además, un mutex de Windows no bloquea al hilo que ya lo tiene, y si su dueño desaparece el siguiente que lo pide continúa. La sección 3 detalla los cuatro argumentos.
- **¿De dónde venía el bloqueo?** Del firmware del dsPIC y del uso del bus SPI desde la PC (sección 4). Esas causas ya estaban identificadas en la propuesta (`docs/propuesta.pdf`) y en la sección "Limitaciones conocidas" del `README.md`.

Los bloques nuevos no tienen mutex porque un solo bloque usa el FT2232H, pero eso es una consecuencia del rediseño, no la corrección del bloqueo.

2. Cómo funcionaban los bloques anteriores

Los bloques anteriores eran tres S-Functions de nivel 2 que compartían el FT2232H. El código está en `SIMULINK/WritePicModificado` y `SIMULINK/ReadPicModificado`, y el modelo que los usa es `MergedPic.slx`. La tabla 1 resume lo que hacía cada uno.

Bloque	Qué hacía	Operación SPI por paso
`initializePic`	Abría el canal 0 de libMPSSE a 1 MHz en modo 0, con `CS` activo en bajo en `ADBUS3`, y creaba el mutex. Su salida era la "llave": la dirección de la estructura compartida `HandleFTDI` expresada como entero de 64 bits.	Ninguna
`escribirPic`	Convertía el voltaje a un código de 16 bits, $13107\,(V + 2.5)$. Si el código no había cambiado respecto al paso anterior, no enviaba nada.	`SPI_Write`, 3 bytes
`leerPic`	Leía 12 bytes y buscaba la secuencia `0xAA 0x55` seguida de 4 bytes de posición.	`SPI_Read`, 12 bytes

Tabla 1: Bloques anteriores.

La estructura compartida está definida en `ftdi_compartido.h` (listado 1). Contiene el *handle* del FT2232H, una bandera que indica si el canal se abrió y el *handle* del mutex. El `#include <mutex>` de la biblioteca estándar de C++ no se usa: el mutex es el de Windows.

```
#include <mutex>        // no se usa
#include <windows.h>
#include "ftd2xx.h"
#include "libmpsse_spi.h"

struct HandleFTDI {
    FT_HANDLE handle;
    bool abierto;
    HANDLE semaforo;    // mutex del nucleo de Windows
};
```

Listado 1: Estructura compartida (`SIMULINK/WritePicModificado/ftdi_compartido.h`).

`initializePic` crea el mutex sin nombre en `mdlStart` (`initializePic.cpp`, línea 52) y lo destruye con `CloseHandle` en `mdlTerminate`, después de enviar `0xAA 0x00 0x00`. Con la conversión de `escribirPic`, el código `0x0000` equivale a $-2.5\,\mathrm{V}$, no a $0\,\mathrm{V}$. `escribirPic` y `leerPic` toman el mutex antes de su operación sobre el bus y lo sueltan al terminar (listado 2).

```
// escribirPic, mdlOutputs
if (!primerEnvio[0] && valor_dac == ultimoValor[0])
    return;                                  // valor repetido: no envia
WaitForSingleObject(hw->semaforo, INFINITE);
bufferTx[0] = 0xAA;
bufferTx[1] = (valor_dac >> 8) & 0xFF;
bufferTx[2] = valor_dac & 0xFF;
SPI_Write(hw->handle, bufferTx, 3, &bytesTransferidos, ...);
ReleaseMutex(hw->semaforo);

// leerPic, mdlOutputs
WaitForSingleObject(hw->semaforo, INFINITE);
status = SPI_Read(hw->handle, Buf, 12, &bytesTransferidos, ...);
ReleaseMutex(hw->semaforo);
for (i = 0; i < bytesTransferidos - 5; i++)
    if (Buf[i] == 0xAA && Buf[i+1] == 0x55) { /* posicion en Buf[i+2..i+5] */ }
```

Listado 2: Uso del mutex en `escribirPic.cpp` (líneas 174–220) y `leerPic.cpp` (líneas 110–125), resumido.

En cada paso de simulación había entonces dos transacciones independientes sobre el bus: una escritura de 3 bytes y una lectura de 12. La parte izquierda de la figura 2 lo muestra.

3. El mutex

3.1. Es compatible con Windows

Las cuatro funciones que usan los bloques, `CreateMutex`, `WaitForSingleObject`, `ReleaseMutex` y `CloseHandle`, son del API de Windows y están en `kernel32.dll`, que está presente en todas las versiones. El mutex se crea sin nombre, por lo que sólo se puede alcanzar a través del *handle* guardado en `HandleFTDI`: ningún otro proceso puede tomarlo. No hay ninguna incompatibilidad entre este mutex y Windows, MATLAB o el compilador MSVC.

3.2. No puede causar el bloqueo

Un mutex sólo detiene a un hilo cuando otro hilo distinto lo tiene tomado y no lo suelta. En estos bloques esa situación no se presenta, por cuatro razones.

1. **Nunca hay dos bloques compitiendo por él.** En una simulación normal, Simulink calcula las salidas de los bloques en un orden fijo y en un solo hilo: llama a `mdlOutputs` de `escribirPic`, regresa, y después llama a `mdlOutputs` de `leerPic` (figura 1). Cuando un bloque pide el mutex, el otro ya lo soltó, y `WaitForSingleObject` regresa de inmediato. El mutex nunca espera.
2. **Un mutex de Windows no bloquea al hilo que ya lo tiene.** Es recursivo para su dueño: el mismo hilo puede volver a tomarlo sin quedarse esperando. Aunque faltara una llamada a `ReleaseMutex` en algún camino del código, el hilo de la simulación no se detendría por ello.
3. **Tampoco espera si su dueño desaparece.** Si el hilo que tiene el mutex termina sin soltarlo, Windows lo marca como abandonado y el siguiente `WaitForSingleObject` regresa con `WAIT_ABANDONED` en lugar de esperar para siempre.
4. **No hay otro candado en la cadena.** Se revisaron las funciones que importa `libmpsse.dll`: de `kernel32` sólo usa `Sleep`, `QueryPerformanceCounter`, `LoadLibraryW`, `GetProcAddress` y funciones de diagnóstico, y carga `ftd2xx.dll` en tiempo de ejecución. No importa ninguna primitiva de sincronización (mutex, sección crítica ni evento). Las lecturas y escrituras del D2XX tienen tiempos límite (`FT_SetTimeouts`), por lo que tampoco esperan indefinidamente.

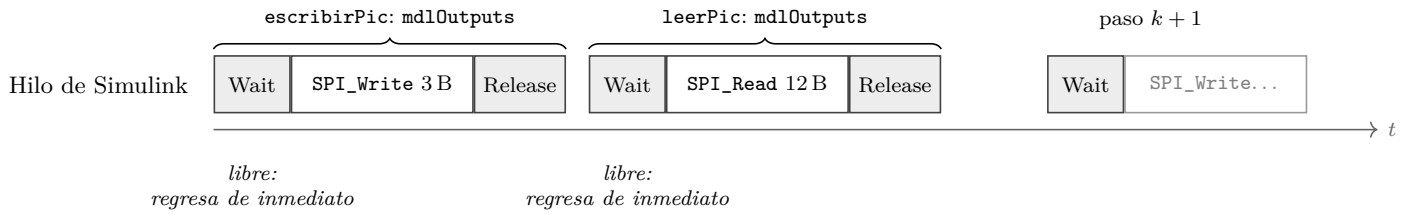


Figura 1: Un paso de simulación con los bloques anteriores. Los dos bloques se ejecutan uno tras otro en el mismo hilo; cada `Wait` encuentra el mutex libre.

3.3. Qué hacía realmente

El mutex sólo evitaba que dos hilos usaran el FT2232H al mismo tiempo, algo que en este modelo no ocurre. Podría ser útil si los bloques se ejecutaran en hilos distintos, por ejemplo en un modelo con varias tareas concurrentes, pero no es el caso de `MergedPic.slx`. No interviene en ninguna de las fallas de la sección 4: con o sin mutex, la PC hacía las mismas dos transacciones en el mismo orden.

4. Las causas reales del bloqueo

El bloqueo venía del firmware anterior del dsPIC (`dsPicController`), que podía quedarse detenido o dejar de recibir, y de la forma en que la PC usaba el bus. La tabla 2 reúne las causas.

Dónde	Problema	Efecto
Firmware	**Espera activa.** La escritura en SPI1 esperaba mientras la FIFO de transmisión estuviera llena, condición que sólo cambia cuando la PC genera reloj.	El lazo principal se detenía y no atendía la recepción.
Firmware	**Desbordamiento sin recuperación.** Con la FIFO de recepción llena se activa `SPIROV`, y el driver generado por MCC no limpiaba la bandera.	SPI1 dejaba de recibir para siempre: el bloqueo permanente.
Firmware	**Respuesta sin sincronía.** El esclavo escribía la posición en la FIFO cada vez que había espacio, sin relación con el momento en que la PC leía.	Datos desfasados y desalineados.
Firmware	**Sin delimitación de tramas.** No usaba `CS`; sólo buscaba `0xAA`, valor que puede aparecer dentro de los datos.	Tramas mal interpretadas.
PC	**Bus full-duplex usado como half-duplex.** `SPI_Write` y `SPI_Read` por separado. Al escribir se descartaba lo que llegaba por MISO; al leer, el dsPIC tomaba los bytes de relleno como datos.	Dos transacciones por paso que se corrompían entre sí y llenaban la FIFO del dsPIC.
PC	**`escribirPic` no reenviaba un valor repetido.**	Si una escritura se perdía, con una entrada constante el error era permanente.
PC	**Al terminar se enviaba `0x0000`.**	$-2.5\,\mathrm{V}$ en lugar de $0\,\mathrm{V}$: el motor quedaba energizado.

Tabla 2: Causas del bloqueo y de las tramas corruptas.

La combinación de las primeras dos explica el bloqueo. En el firmware anterior, el lazo principal escribía los 6 bytes de la respuesta con `SPI1_ByteWrite`, que espera mientras `SPITBF` indique la FIFO de transmisión llena. Durante esa espera nadie leía la FIFO de recepción de 4 niveles, a la que llegaban los 3 bytes de cada escritura y los 12 de relleno de cada lectura de la PC. Al llenarse se activaba `SPIROV` y, como ninguna parte del código lo borraba, SPI1 ya no recibía nada más. Ninguno de estos pasos depende del mutex.

En la primera prueba en Windows (sección 9) apareció otra causa, ajena al firmware: en esa PC libMPSSE numeró la interfaz B del FT2232H como canal 0. Los bloques abrían el canal 0, así que hablaban por la interfaz que no está cableada al dsPIC y sólo leían `0xFF`. Los bloques nuevos buscan la interfaz por la que responde el dsPIC (sección 7), de modo que ya no dependen de esa numeración.

5. El protocolo nuevo

Cada paso de simulación hace una sola transferencia full-duplex de 8 bytes con `SPI_ReadWrite`, con `CS` en bajo durante toda la trama. Mientras la PC envía el voltaje por MOSI, el dsPIC devuelve la posición por MISO en los mismos ocho bytes de reloj. El formato está definido en un solo archivo, `protocolo/daq_protocolo.h`, que comparten el firmware y la PC (tabla 3).

Byte	PC → dsPIC (MOSI)	dsPIC → PC (MISO)
0	Encabezado `0xA5`	Encabezado `0x5A`
1	Número de secuencia	Eco de la secuencia de la última trama válida
2	Código del DAC, byte alto	Posición del encoder, `int32`
3	Código del DAC, byte bajo	little-endian (bytes 2 a 5)
4	Banderas: bit 0 salida habilitada, bit 1 reinicio del encoder	
5	Reservado (0)	
6	Reservado (0)	Status: errores de CRC, encabezado, longitud y vigilancia; causa de reinicio en los 4 bits altos
7	CRC-8 de los bytes 0 a 6	CRC-8 de los bytes 0 a 6

Tabla 3: Formato de la trama de 8 bytes.

Qué resuelve cada parte:

- **Una transferencia por paso.** Ya no hay una escritura y una lectura separadas que se estorben: cada byte que entra al dsPIC es parte de una trama válida y cada byte que sale es parte de la respuesta.
- **`CS` delimita la trama.** El dsPIC sabe dónde empieza y termina sin buscar un valor especial en los datos.
- **Secuencia y CRC-8** (polinomio `0x07`, CRC-8/SMBUS). La PC detecta tramas dañadas, perdidas o repetidas, y las cuenta como errores.
- **El voltaje se envía en cada paso**, aunque no cambie, de modo que una trama dañada sólo afecta un periodo.
- **Bandera de salida habilitada.** Con ella en 0, el dsPIC lleva el DAC a 0 V (código 32767). Al terminar la simulación se envía una trama así, en lugar de -2.5 V.
- **Status.** El dsPIC informa sus propios errores y, una vez después de cada arranque, la causa del reinicio (encendido, MCLR, *watchdog* o tipo de trampa).

La respuesta de la transferencia k se arma al terminar la $k-1$, por lo que la posición llega con un paso de retraso. Sumado al paso en que se aplica el voltaje, el retardo total del lazo es de dos periodos de muestreo.

6. El firmware del dsPIC: `dsPicDaq`

El firmware nuevo ya no tiene esperas activas sobre el SPI y recupera el desbordamiento en cada trama, que eran las dos causas del bloqueo. El proyecto está en `MPLAB/dsPicDaq.X` y la imagen compilada en `MPLAB/dsPicDaq.hex`.

- **DMA en lugar de espera activa.** SPI1 trabaja como esclavo en modo de buffer estándar. DMA0 recibe y DMA1 transmite durante la transferencia, sin intervención del lazo principal. La FIFO del modo mejorado no se usa: tiene 4 niveles de 8 bits, menos que una trama.
- **Fin de trama por `CS`.** `CS` (RB10) se asigna con PPS a `SS1` y a `INT1`. En el flanco de subida, la interrupción valida la trama, reinicia SPI1 (vacía los buffers y borra `SPIROV`), muestrea el QEI, arma la respuesta siguiente y reprograma el DMA. Aunque una trama llegue incompleta o dañada, la siguiente empieza con el módulo limpio.

- **DAC.** El lazo principal sólo escribe al DAC8554 cuando el valor cambia.
- **Vigilancia.** Timer1: si pasan 50 ms sin una trama válida, el DAC va a 0 V y se reporta en el status. Si Simulink se detiene, el motor no se queda con el último voltaje.
- **Reloj.** El PLL eleva la frecuencia de instrucción de 4 MHz a $F_{cy} = 50$ MHz. Con 4 MHz, la interrupción de fin de trama tardaba cerca de 100 μs y el esclavo no alcanzaba a presentar a tiempo el primer bit de cada byte a 1 MHz.
- **Causa de reinicio.** `reinicio.c` guarda el código de una trampa en RAM persistente antes del reinicio, y el dsPIC lo reporta en la primera respuesta después de arrancar.
- **Medición.** LED0 (RD10) queda en alto durante la interrupción, para medir su duración con un analizador lógico.

El firmware se compila con `-O1` para acortar la interrupción de fin de trama, que fija el intervalo mínimo entre tramas. En Linux se compila sin MPLAB X con `MPLAB/dsPicDaq.X/compilar.sh`.

7. Los bloques nuevos de Simulink

Hay tres juegos de bloques en `SIMULINK/DaqPic`, y en todos un solo bloque usa el FT2232H, una vez por paso. Por eso ya no hace falta el mutex. Se compilan todos con `compilar.m` (tabla 4).

Juego	Bloques	Uso
Un bloque	`daqPic`: entra el voltaje; salen posición, errores y atrasos	Modelos nuevos
Tres bloques	`daqPicInicio` (llave), `daqPicEscribir` (llave, voltaje), `daqPicLeer` (posición, errores, atrasos)	Modelos que separan inicialización, escritura y lectura
Nombres anteriores	`initializePic`, `escribirPic`, `leerPic`, con los mismos puertos y en el mismo orden	Usar `MergedPic.slx`, `ReadPicModificado.slx` y `WritePicModificado.slx` sin modificarlos

Tabla 4: Bloques nuevos.

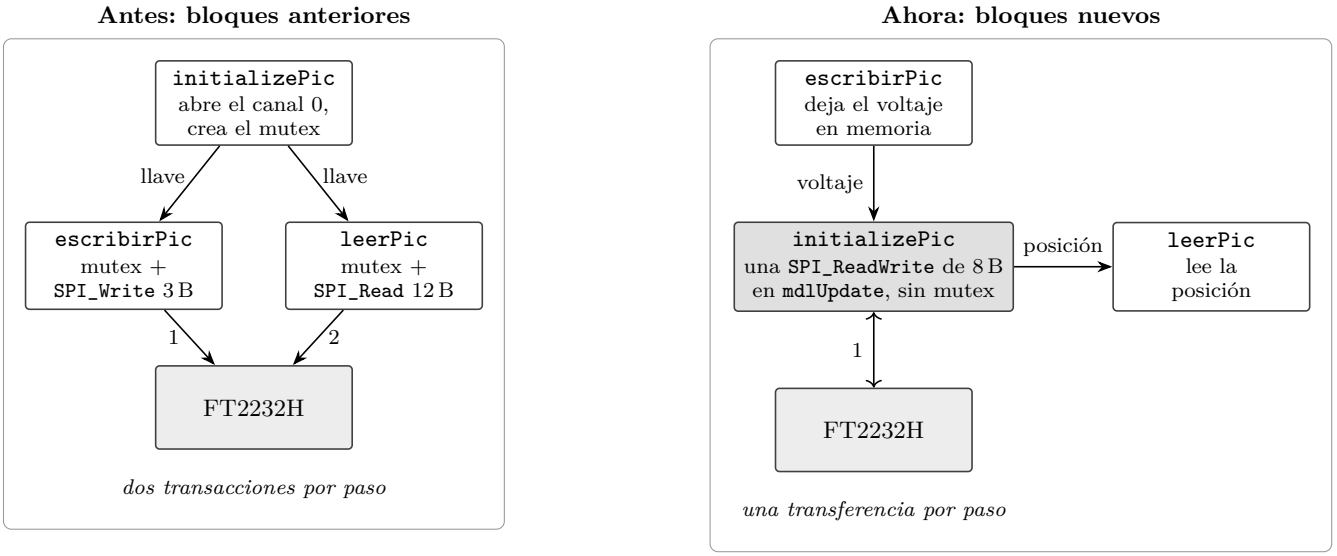


Figura 2: Operaciones sobre el FT2232H en cada paso, con los bloques anteriores y con los nuevos de los mismos nombres. En los nuevos, `escribirPic` y `leerPic` sólo leen y escriben memoria; `initializePic` es el único que usa el bus, así que no queda nada que proteger con un mutex.

Cómo trabajan los de tres bloques:

- `escribirPic` y `leerPic` ya no usan el bus. `escribirPic` deja el voltaje en la memoria compartida y `leerPic` lee la última posición recibida.
- `initializePic` hace la única transferencia del paso, en su `mdlUpdate`. Simulink calcula las salidas de todos los bloques antes de cualquier actualización, así que el voltaje del paso ya está escrito y el orden entre bloques no importa.
- La llave sigue siendo la dirección del estado compartido, ahora como `double`. Antes de usarla, cada bloque comprueba con `VirtualQuery` que la dirección sea válida y que el estado tenga una marca conocida. Una señal equivocada produce un mensaje de error en lugar de cerrar MATLAB.

Otros cambios respecto a los bloques anteriores:

- **Búsqueda de la interfaz.** Se prueba primero el canal cuyo número de serie termina en `A` y después los demás, con tramas que dejan la salida en 0 V y no tocan el encoder; se usa el primero por el que el dsPIC responde con una trama válida (`protocolo/daq_canal.h`). Funciona aunque libMPSSE numere las interfaces en otro orden y aunque el dsPIC se cablee a la interfaz B. Si no responde por ninguna, el mensaje de error indica qué se leyó en cada una, por ejemplo “interfaz A: solo 0xFF; interfaz B: solo 0xFF”.
- **Verificación al arrancar.** El bloque pone en cero el encoder y comprueba que el dsPIC responda con el protocolo nuevo. Si no, detiene la simulación con un mensaje claro.
- **Tiempo real.** Cada paso espera al reloj de la PC (`QueryPerformanceCounter`) y cuenta como atraso el que llega más de un periodo tarde.
- **Errores con `ssSetErrorStatus`** en lugar de `mexErrMsgTxt`, y `mdlTerminate` tolera que Simulink lo llame sin haber llamado antes a `mdlStart`.

- **Al terminar**, una trama con la salida deshabilitada (0 V) y, en `initializePic`, un resumen en la ventana de comandos con pasos, errores y atrasos.

Para que MATLAB use los bloques nuevos y no los anteriores, la simulación se ejecuta con `SIMULINK/DaqPic` como carpeta actual. El paquete para la carpeta del asesor lo arma `herramientas/asesor/empaquetar.py`.

8. Herramientas de prueba sin Simulink

Para separar las fallas del enlace de las de Simulink, el enlace se probó primero sin MATLAB. Las herramientas están en `herramientas/` y comparten el núcleo de la prueba (`herramientas/comun/enlace_prueba.h`).

Herramienta	Qué hace
`protocolo/prueba_protocolo.c`	Pruebas del formato de trama y del CRC, sin hardware, con un modelo del esclavo que reproduce el retraso de una trama.
`prueba_enlace`	Envía tramas al dsPIC y cuenta errores de encabezado, CRC y secuencia, los que reporta el dsPIC y la duración de cada transferencia. Compila en Windows con libMPSSE (`compilar.bat`) y en Linux con libftdi1. Con `-lazo` une MOSI y MISO dentro del FT2232H para probar la PC sin el dsPIC.
`reporte/reporte.py`	Genera un PDF a partir del registro CSV de una prueba: errores, histograma de duraciones y mapa de bits erróneos.
`visor_enlace`	Interfaz gráfica en tiempo real que ejecuta el mismo lazo que un modelo de Simulink con `daqPic` (lazo abierto o PID), con osciloscopio, contadores y la tabla de tramas con error. Tiene un modo simulado sin hardware.
`arnes_simulink`	Ejecuta las S-Functions reales en Linux, sin MATLAB, llamándolas en el orden de Simulink contra la tarjeta. Revisa puertos, periodos, llaves inválidas, errores y la liberación del estado.
`bateria_pruebas.sh`	Ejecuta en secuencia el lazo interno y el dsPIC, lo más rápido posible y a 1 kHz, y genera los reportes.

Tabla 5: Herramientas de prueba.

En Linux, la EEPROM del FT2232H está reprogramada con el identificador `2099:0001`, y `herramientas/prueba_enlace/70-daq-spi.rules` da acceso al dispositivo sin `sudo`. El procedimiento completo está en `PRUEBAS.md`.

9. Resultados

Con el firmware nuevo, el PLL y el cable de GND entre las tarjetas, el enlace transmitió 100 000 tramas sin errores, tanto a 1 kHz como lo más rápido posible. En Windows, `MergedPic.slx` se ejecutó sin modificaciones y sin errores. Cada prueba está en `resultados/<fecha>`, con su registro CSV y su reporte en PDF (tabla 6).

Fecha	Prueba	Resultado
2026-10-07	Windows 10, MATLAB R2025b: `MergedPic.slx` sin modificaciones con los bloques nuevos, 10 001 pasos a 1 kHz	0 errores, 1 atraso
2026-10-07	Windows: `DaqPicPrueba` y `DaqPicBloquesPrueba` con el modelo abierto en el editor	4 a 10 errores por vigilancia: Simulink se detuvo cada 0.25 s y algunas pausas pasaron de 50 ms
2026-10-06	Rampa de ± 1 V, 10 000 tramas a 1 kHz	0 errores en el enlace; el DAC no respondió porque DIN y SYNC estaban en terminales equivocadas
2026-10-06	Con el cable de GND: 100 000 tramas a 1 kHz y 100 000 a 7408 tramas/s	0 errores
2026-10-06	Sin cable de GND, ambas tarjetas en el mismo concentrador USB	0.025 % de las transferencias con error
2026-10-05	Con PLL, sin cable de GND, cada tarjeta en un puerto USB distinto	0.7 % con error, por pulsos falsos en `CS`
2026-10-05	Sin PLL ($F_{cy} = 4$ MHz), 100 000 tramas a 1 kHz	83 670 errores de CRC: el dsPIC no alcanzaba a presentar el primer bit de cada byte

Tabla 6: Resultados de las pruebas, de la más reciente a la más antigua.

El resultado de `MergedPic.slx` es el que responde a la pregunta en la práctica: el mismo modelo, con los mismos tres bloques y en el mismo Windows, dejó de fallar al cambiar el firmware y el uso del bus, no al quitar el mutex.

Dos hallazgos de estas pruebas no tienen que ver con el software:

- **El cable de GND entre la Curiosity Nano y la tarjeta del FT2232H es necesario.** Sin él aparecen pulsos falsos en `CS`, y el dsPIC rearma la respuesta a mitad de la trama.
- **El cableado del DAC8554 estaba mal.** DIN estaba en RB7 en lugar de RB2 (SDO2) y SYNC en una terminal vecina de RB4. Por eso, en las pruebas con la salida habilitada, el motor no se movió y la posición se quedó en 0.

10. Pendientes

El enlace y los bloques ya funcionan en Windows; falta la salida analógica y el motor.

1. Corregir el cableado del DAC8554 (DIN a RB2, SYNC a RB4) y medir `VOUTA` con `prueba_enlace -voltaje`.
2. Probar con el motor conectado: primero en lazo abierto y después con el PID de `crear_modelo_pid.m`.
3. Comprobar en la interfaz gráfica de Simulink, no sólo con `matlab -batch`, si hay pausas de más de 50 ms con el modelo abierto.
4. Ejecutar `MergedPic.slx` en la PC del laboratorio con el paquete de `herramientas/asesor`.